
GRC Control Validator

Architecture and Workflow Reference

Subtitle: Complete Pipeline Documentation | Version 2.0

Author note: Auto-generated from repository source | June 2026

Audience: Developers, Compliance Engineers, Security Architects

Document type: Technical Architecture & Workflow Reference

Abstract

This document describes the end-to-end architecture of the GRC Control Validator, a Python/Streamlit application for ingesting, normalizing, routing, and deterministically evaluating GRC control statements against compliance frameworks. It covers all pipeline stages, module responsibilities, circuit breaker behavior, scoring methodology, and rationale generation. Intended for developers contributing to or integrating with the repository.

Table of Contents

- 1. Repository Overview
- 2. Pipeline Architecture Diagram
- 3. Stage-by-Stage Deep Dive
 - 3.1 Stage 1 — Multi-Format Ingestion
 - 3.2 Stage 2 — Sanitization Pipeline
 - 3.3 Stage 3 — Transformer Normalization
 - 3.4 Stage 4 — Semantic Routing
 - 3.5 Stage 5 — Deterministic Evaluation Engine

- 3.6 Stage 6 — Circuit Breakers
- 3.7 Stage 7 — Compliance Scoring and Grading
- 3.8 Stage 8 — Rationale Generation and Output
- 4. Streamlit UI Behavior and `st.stop()` Pattern
- 5. Data Flow Summary Table
- 6. Extending the System
- 7. Environment and Dependencies
- 8. Known Limitations and Future Roadmap

1. Repository Overview

The GRC Control Validator repository is organized around a single-responsibility module architecture. Each Python file owns one distinct layer of the processing pipeline, enabling independent testing, substitution, and extension. The following table describes the purpose and role of each core file.

File / Asset	Role	Key Responsibilities
<code>app.py</code>	Streamlit Front-End & Orchestration Entry Point	Manages file upload UI, triggers the full pipeline on user action, displays results panels, and enforces graceful halts via <code>st.stop()</code> on all fault conditions to prevent partial renders.
<code>sanitize.py</code>	Input Sanitization Pipeline	Strips unsafe characters, normalizes whitespace, removes injection artifacts (e.g., <code>ignore previous instructions</code> , <code>system: prefixes</code>), validates text length and UTF-8 encoding before any downstream processing occurs.
<code>transform.py</code>	Transformer-Based Normalization	Applies text normalization (lowercasing, punctuation handling, optional stopwords filtering), generates a sentence embedding vector using a configurable

File / Asset	Role	Key Responsibilities
		transformer model (e.g., all-MiniLM-L6-v2), and returns a structured payload dict.
<code>orchestrate.py</code>	Central Pipeline Orchestrator	Coordinates calls across all pipeline stages — ingestion, sanitization, transformation, routing, evaluation, scoring, and output. Owns all circuit breaker guard functions and manages error propagation state.
Mapping Files (JSON/CSV)	RAG-Ready Semantic Datasets	Link control keywords, framework identifiers (NIST, ISO 27001, SOC 2, HIPAA, PCI-DSS, CIS Controls), and semantic intent clusters to centroid vectors. Used by the semantic router to assign framework context without any LLM call. Extensible without code changes.
<code>README.md</code>	Repository Entry Point	Provides setup instructions, dependency installation guidance, environment variable configuration, and usage examples for first-time contributors and integrators.

① Design Principle

Each module is independently importable and testable. The orchestrator (`orchestrate.py`) is the only file that holds references to all other modules. No module imports another directly, except through orchestrator-mediated calls.

2. Pipeline Architecture Diagram

The following ASCII diagram illustrates the complete data flow through all eight stages of the GRC Control Validator pipeline. Each stage is a discrete processing boundary with defined inputs, outputs, and fault conditions guarded by circuit breakers.



3. Stage-by-Stage Deep Dive

Each of the following subsections describes one pipeline stage in full detail, including its purpose, responsible module, inputs and outputs, key logic, and error handling behavior.

3.1 Stage 1 — Multi-Format Ingestion

Module: `orchestrate.py` → `ingest()` dispatcher

Input: Raw uploaded file bytes + file extension

Output: Single unified raw text string

Purpose: Accept uploaded documents in any supported format and extract a single, consistent raw text string for downstream processing. The ingestion layer is the only point in the pipeline that handles binary file data.

- Supports `.txt` (direct UTF-8 read), `.pdf` (page-by-page text extraction using PyMuPDF or pdfplumber), and `.docx` (paragraph-by-paragraph extraction via python-docx).
- Format dispatch is handled by `orchestrate.py`'s `ingest()` function, which routes to the appropriate parser based on the file extension string.
- All parsers strip document metadata, headers, footers, and formatting artifacts during extraction, so only semantic body text is passed downstream.
- If the uploaded file has an extension not in the supported formats list, the dispatcher raises a typed `IngestionError`, which is immediately caught by the unsupported format circuit breaker.

- All format parsers return the same output type — a plain Python `str` — ensuring no downstream module needs to be aware of the original file format.
- Multi-page PDFs are concatenated with a newline separator to preserve paragraph boundaries across page breaks.



Error Condition

An unsupported file format (e.g., `.xlsx`, `.png`) raises `IngestionError: unsupported_format`. The circuit breaker catches this, sets the fault state, and signals `app.py` to call `st.stop()`. The user sees: *"File format not supported. Please upload a .txt, .pdf, or .docx file."*

3.2 Stage 2 — Sanitization Pipeline

Module: `sanitize.py`

Input: Raw text string from Stage 1

Output: Sanitized, validated text string; or raises `SanitizationError`

Purpose: Eliminate unsafe, malformed, or adversarial content from the raw text before it enters any processing logic, ensuring all downstream stages operate on well-formed, predictable input.

- Removes all HTML and XML tags using regex stripping to eliminate any embedded markup that survived ingestion.
- Strips null bytes (`\x00`), non-printable control characters, and other binary artifacts that can destabilize text processing libraries.
- Collapses repeated whitespace (multiple spaces, tabs, runs of newlines) into single normalized equivalents; standardizes line endings to `\n`.

- Enforces UTF-8 encoding — any character that cannot be represented in UTF-8 is replaced with the Unicode replacement character and a warning is logged.
- Validates text length: text shorter than the minimum threshold triggers the empty content guard; text exceeding the maximum limit is truncated with a user-visible warning appended to the result metadata.
- Detects and removes prompt injection patterns, including phrases such as `ignore previous instructions`, `system:`, `###SYSTEM`, and similar adversarial prefixes that could interfere with downstream template logic.



Error Condition

If post-sanitization text is empty or consists only of whitespace, `SanitizationError: empty_content` is raised. The empty content circuit breaker halts the pipeline and surfaces: *"No readable text content was found after sanitization. Please verify your file."*

3.3 Stage 3 — Transformer Normalization

Module: `transform.py`

Input: Sanitized text string from Stage 2

Output: Structured payload dict: `{"normalized_text": str, "embedding": List[float], "token_count": int}`

Purpose: Convert the sanitized text into both a normalized string form (for rule-based comparison) and a semantic embedding vector (for similarity-based routing), packaged as a single structured payload.

- Applies deterministic text normalization in sequence: lowercase conversion, punctuation stripping, and optional stopword filtering (configurable via a stopword list constant in `transform.py`).
- Normalized text is used exclusively for rule-based evaluation in Stage 5 — it is not passed to any external service.
- The transformer model (default: `all-MiniLM-L6-v2` or equivalent) is loaded once at application startup, not per-request, to avoid latency on every validation run.
- The embedding vector is a fixed-length list of floats matching the dimensionality of the mapping file centroid vectors (must be consistent — see Section 6 on model swapping).
- Token count is computed and stored in the payload for the token limit circuit breaker check that follows this stage.
- The model name is controlled by the `MODEL_NAME` constant in `transform.py`, making model substitution a one-line change.

Payload Schema

`{"normalized_text": str, "embedding": List[float], "token_count": int}` — this dict is the contract between Stage 3 and all subsequent stages. Any future field additions should be backward-compatible and optional.

3.4 Stage 4 — Semantic Routing

Module: `orchestrate.py` (routing logic) + JSON/CSV mapping files

Input: Structured payload dict from Stage 3

Output: Framework label (string) + intent classification (string)

Purpose: Assign each control statement to the most semantically appropriate compliance framework and classify its control intent type — entirely without any LLM call, using deterministic cosine similarity against pre-computed centroid vectors.

- RAG-ready mapping files (JSON/CSV) are loaded once at startup into memory; they are not re-read per request. Loading is triggered by `orchestrate.py`'s initialization sequence.
- Cosine similarity is computed (using NumPy) between the control's embedding vector and each framework's centroid vector stored in the mapping files.
- The framework with the highest similarity score is assigned, provided it meets or exceeds the configurable threshold (default: 0.60). Supported frameworks: NIST SP 800-53, ISO 27001, SOC 2, HIPAA, PCI-DSS, CIS Controls.
- A secondary classification assigns the control's intent type: **preventive** (stops an incident from occurring), **detective** (identifies when an incident has occurred), **corrective** (restores system state after an incident), or **administrative** (policy-level governance controls).
- Mapping datasets are framework-agnostic and extensible: adding a new framework requires only a new entry in the mapping files with keyword clusters and centroid vectors — no code changes are required.
- If no framework exceeds the similarity threshold, the ambiguous framework circuit breaker fires immediately.



Error Condition

If the highest cosine similarity across all frameworks is below 0.60, the ambiguous framework guard triggers: *"This control statement could not be confidently mapped to a known compliance framework. Please refine the control language or check the mapping files."*

3.5 Stage 5 — Deterministic Evaluation Engine

Module: `orchestrate.py` (coordinates evaluation criteria per framework)

Input: Structured payload dict + framework label + intent classification

Output: Evaluation dict with per-criterion boolean flags and weighted sub-scores

Purpose: Apply a fully rule-based, deterministic checklist evaluation against the criteria defined for the assigned compliance framework, producing per-dimension scores and boolean pass/fail flags for each criterion — with no LLM calls at this stage.

- **Obligation Strength:** Detects presence and strength of obligation language — `shall` (strongest), `must`, `will`, `should` (weakest), or absent. Weighted accordingly.
- **Specificity:** Checks for quantified thresholds (e.g., numeric values, time periods), named systems or assets, and actionable verb constructs that indicate a specific rather than vague control statement.
- **Completeness:** Validates presence of structural markers addressing: *who* (responsible party), *what* (the action or constraint), *when* (frequency or trigger), and *how* (mechanism or method).
- **Framework Alignment:** Compares keyword density against the assigned framework's keyword cluster to verify the control's vocabulary matches framework expectations.
- **Control Type Accuracy:** Cross-checks whether the control's linguistic structure matches the classified intent type (e.g., a preventive control should express a prohibition or constraint, not a detection event).
- Each dimension produces a sub-score (float, 0.0–1.0) and a boolean flag. The full evaluation dict is passed intact to Stage 7 for weighted aggregation and to Stage 8 for rationale generation.

3.6 Stage 6 — Circuit Breakers

Module: `orchestrate.py` (guard functions) → `signals app.py`

Input: Payload dict at each inter-stage boundary

Output: `(True, "")` on pass; `(False, "fault_message")` on halt

Purpose: Enforce deterministic quality gates between every pipeline stage, ensuring no malformed, ambiguous, or low-confidence payload propagates further into the pipeline.

- Implemented as standalone guard functions in `orchestrate.py`, each with the signature `(payload: dict) → (bool, str)`. Returns `(True, "")` on pass, or `(False, "fault message")` on halt.
- All circuit breaker conditions: embedding confidence below `0.50`; sanitized text empty or whitespace-only; file format not in supported extensions list; no framework similarity above `0.60`; total token count exceeds maximum limit.
- On halt: the guard function logs the fault condition with a structured log entry, populates an error state dict (fault code + human-readable reason + remediation hint), and returns the halt signal to `orchestrate.py`.
- `orchestrate.py` propagates the error state dict to `app.py`, which immediately calls `st.stop()` — halting all further Streamlit script execution and rendering only the error callout.
- Circuit breakers are stateless and idempotent: they do not modify the payload, hold no internal state between calls, and are safe to re-invoke after the user corrects their input and re-submits.
- Every fault message exposed to the user includes both a plain-language description of the fault and a specific remediation hint (e.g., what to change in the uploaded file, or what threshold was not met).

3.7 Stage 7 — Compliance Scoring and Grading

Module: `orchestrate.py` (scoring aggregation)

Input: Evaluation dict from Stage 5

Output: `{"score": float, "grade": str, "dimension_scores": dict}`

Purpose: Aggregate per-dimension sub-scores into a single deterministic composite score on a 0–100 scale, then apply grade band classification to produce a final compliance verdict.

- Applies the `DIMENSION_WEIGHTS` constant (stored in the scoring module or `orchestrate.py`) to produce a weighted composite score.
- Default weights: Obligation Strength 25%, Specificity 25%, Completeness 20%, Framework Alignment 20%, Control Type Accuracy 10%. All weights must sum to 1.0.
- Grade bands: **PASS** (score ≥ 75), **REVIEW** (score 50–74), **FAIL** (score < 50).
- Scoring is fully deterministic — identical input text will always produce the identical score, grade, and dimension breakdown. No randomness, sampling, or probabilistic elements are present.
- The per-dimension breakdown is preserved in the output dict for granular display in the Streamlit results panel and for inclusion in the JSON export.

Grade	Score Range	Interpretation	UI Badge Color
PASS	75 – 100	Control meets framework standards with sufficient obligation, specificity, and completeness.	Green (#155724)
REVIEW	50 – 74	Control partially meets standards. One or more dimensions require strengthening before formal	Amber (#856404)

Grade	Score Range	Interpretation	UI Badge Color
		compliance use.	
FAIL	0 – 49	Control does not meet minimum standards. Significant gaps in obligation language, specificity, or completeness.	Red (#721c24)

3.8 Stage 8 — Rationale Generation and Output

Module: `orchestrate.py` (rationale templates) → `app.py` (rendering)

Input: Evaluation dict + score dict + framework label

Output: Rendered Streamlit result panel + JSON export dict

Purpose: Transform the structured evaluation output into human-readable verdict statements, gap descriptions, and remediation suggestions — all via deterministic templates, with no LLM generation — and render the full result in the Streamlit UI.

- For each evaluated criterion that received a failing or partial flag, a gap description string is generated from a pre-defined template keyed on the criterion name and the assigned framework.
- Each gap description is paired with a specific remediation suggestion.
Example: *"This control lacks obligation language. Consider replacing 'should consider' with 'shall' or 'must' to meet NIST SP 800-53 requirement strength standards."*
- All rationale strings are template-based and deterministic — there is no LLM involved. The same failing criterion always produces the same rationale text, making output auditable and reproducible.

- The Streamlit result panel renders: a color-coded score badge (PASS/REVIEW/FAIL), the composite score value, a dimension breakdown table, and an expandable rationale list (one entry per evaluated criterion).
 - The complete result — including score, grade, dimension scores, evaluation flags, framework assignment, intent classification, and all rationale strings — is serialized to a JSON export dict, accessible via a download button in the Streamlit UI.
 - No LLM API keys or external network calls are required at this stage or any other stage. The application is fully offline-capable once the embedding model is downloaded at first run.
-

4. Streamlit UI Behavior and `st.stop()` Pattern

The Streamlit front-end (`app.py`) is the user-facing entry point and the terminal renderer for all pipeline results. Its behavior is designed to be predictable, fault-transparent, and safe — never rendering partial or misleading output.

4.1 File Upload and Trigger

- The file uploader (`st.file_uploader`) accepts `.txt`, `.pdf`, and `.docx` file types only. On upload, the file name and size are displayed immediately as metadata.
- Pipeline execution is triggered by a **"Validate Control"** button click — the pipeline does not run automatically on upload, giving the user a moment to confirm their selection.
- A `st.spinner("Validating control statement...")` context manager wraps the full pipeline call, providing visual feedback during processing.

- Session state (`st.session_state`) is used to persist the last successful validation result, so it survives Streamlit reruns triggered by other UI interactions without re-running the pipeline.

4.2 The `st.stop()` Guard Pattern

The `st.stop()` call is the correct and idiomatic Streamlit pattern for guard clauses at the top level of a Streamlit script. When invoked, it immediately halts all further execution of the current script run — no code below the call executes, and no further UI elements are rendered.



Why `st.stop()` and not `return`?

Streamlit scripts execute as a single flat module, not inside a function that can be returned from. Using `return` at the top level is a `SyntaxError`. `st.stop()` is the correct mechanism: it raises a Streamlit-internal `StopException` that the runtime catches cleanly, finalizing the current render pass without executing any remaining statements.

- Any circuit breaker fault signal received from `orchestrate.py` causes `app.py` to render an error callout with the fault reason and remediation hint, then immediately call `st.stop()`.
- This guarantees that no result panel, score badge, or rationale content is ever rendered below an error message — eliminating any risk of a user interpreting a partial result as a valid compliance verdict.
- After a fault, the user corrects their input (e.g., uploads a different file) and clicks "Validate Control" again. The pipeline re-runs from scratch; circuit breakers are stateless and handle this cleanly.

4.3 Success Path Rendering

- On a successful pipeline run, the result panel renders: a color-coded grade badge (green / amber / red), the composite score (0-100), a dimension breakdown table, and a collapsible rationale section with per-criterion gap descriptions and remediation suggestions.
- A **Download JSON** button exports the full result dict for downstream system integration, audit logging, or bulk processing workflows.
- No LLM API keys are required. The application runs entirely offline once the sentence-transformer model has been downloaded to the local model cache on first startup.

5. Data Flow Summary Table

The following table summarizes all eight pipeline stages in a single reference view, covering module ownership, input/output contracts, and error conditions.

Stage	Module	Input	Output	Error / Circuit Breaker Condition
1 — Ingestion	<code>orchestrate.py</code> → <code>ingest()</code>	Uploaded file bytes + extension	Unified raw text string	<code>IngestionError</code> : <code>unsupported_format</code> — unsupported file extension. Circuit breaker halts.
2 — Sanitization	<code>sanitize.py</code>	Raw text string	Sanitized, validated text string	<code>SanitizationError</code> : <code>empty_content</code> — text is empty or whitespace-only after sanitization. Circuit breaker halts.
3 —	<code>transform.py</code>	Sanitized text	Payload dict:	Embedding

Stage	Module	Input	Output	Error / Circuit Breaker Condition
Normalization		string	normalized text + embedding vector + token count	confidence < 0.50 or token count exceeds limit — circuit breaker halts.
4 — Semantic Routing	<code>orchestrate.py</code> + mapping files	Payload dict (embedding vector)	Framework label (string) + intent classification (string)	No framework similarity ≥ 0.60 — ambiguous framework circuit breaker halts.
5 — Evaluation	<code>orchestrate.py</code>	Payload dict + framework label + intent type	Evaluation dict: per-criterion boolean flags + sub-scores	Aggregate confidence below threshold — score guard circuit breaker halts.
6 — Circuit Breakers	<code>orchestrate.py</code> (guard functions)	Payload dict at each stage boundary	(True, "") on pass; (False, msg) on halt	Any fault condition — logs fault, sets error state, signals <code>app.py</code> to call <code>st.stop()</code> .
7 — Scoring	<code>orchestrate.py</code> (aggregation)	Evaluation dict from Stage 5	<code>{"score": float, "grade": str, "dimension_scores": dict}</code>	None at this stage — circuit breaker precedes it. Score is always producible if Stage 5 completes.
8 — Output	<code>orchestrate.py</code> (templates) + <code>app.py</code> (render)	Evaluation dict + score dict + framework label	Streamlit result panel + JSON export dict	None — this stage only executes after all circuit breakers have passed.

6. Extending the System

The GRC Control Validator is designed for extensibility. The following subsections describe the correct approach for each category of extension, with notes on which files to modify and what invariants must be preserved.

6.1 Adding a New Compliance Framework

No code changes are required to add a new compliance framework to the routing layer.

1. Open the mapping file (JSON or CSV) in the repository's `mappings/` directory.
2. Add a new entry with the framework identifier, its keyword cluster (array of representative terms), and its centroid vector (pre-computed embedding of a representative control corpus).
3. Restart the application — `orchestrate.py` loads the mapping file at startup, and the new framework will immediately be available to the cosine similarity router.
4. Optionally, add framework-specific evaluation criteria to the evaluation engine if the new framework has unique criterion requirements beyond the default five dimensions.

6.2 Adding a New File Format

5. Implement a new parser function (e.g., `parse_rtf(file_bytes: bytes) → str`) in a module accessible to `orchestrate.py`.
6. Register the new file extension in the `SUPPORTED_FORMATS` constant in `orchestrate.py`.
7. Add a dispatch branch in the `ingest()` function to call the new parser when the new extension is detected.
8. Add the new extension to the `type` parameter of the `st.file_uploader` call in `app.py`.

6.3 Adjusting Scoring Weights

- Edit the `DIMENSION_WEIGHTS` constant in the scoring module (or in `orchestrate.py` if co-located).
- The dict must contain exactly five keys corresponding to the five evaluation dimensions, and the values must sum to exactly `1.0`. A startup assertion validates this invariant.
- Example: to increase the weight of Obligation Strength at the expense of Control Type Accuracy, adjust those two values while keeping the sum at `1.0`.

6.4 Adding a New Circuit Breaker Condition

Follow the existing guard function pattern exactly:

- Define a new function in `orchestrate.py` with the signature `(payload: dict) → tuple[bool, str]`.
- Return `(True, "")` when the payload passes the condition; return `(False, "human-readable fault message with remediation hint")` when it should halt.
- Register the new guard function in the pipeline sequence at the appropriate inter-stage position in the main orchestration flow.
- The guard must be stateless — it must not modify the payload or hold any persistent state between invocations.

6.5 Swapping the Embedding Model

- Change the `MODEL_NAME` constant in `transform.py` to the new model's identifier (e.g., a Hugging Face model name).
- **Critical:** If the new model outputs a different vector dimensionality than the current model, the mapping file centroid vectors must be regenerated using the new model before the application will produce valid similarity scores.
- Regenerate centroids by running the provided centroid generation script against your control corpus with the new model loaded, and replacing the centroid vectors in the mapping files.

- Mismatch between embedding dimensionality and mapping file centroid dimensionality will cause a `ValueError` during cosine similarity computation, which is caught and surfaced as a startup configuration error.

7. Environment and Dependencies

The GRC Control Validator requires Python 3.10 or later and a standard virtual environment setup. No external API keys, cloud services, or network access are required at runtime beyond the initial model download.

Package	Version Constraint	Purpose
<code>streamlit</code>	≥ 1.30	Web UI framework — file upload, result rendering, session state, <code>st.stop()</code> guard pattern.
<code>sentence-transformers</code>	≥ 2.6	Sentence embedding generation. Downloads the configured model (<code>all-MiniLM-L6-v2</code> default) on first run. Offline-capable thereafter.
<code>pdfplumber</code> or <code>PyMuPDF</code>	Latest stable	PDF text extraction in Stage 1. Either library is supported; configure the preferred one in <code>orchestrate.py</code> .
<code>python-docx</code>	≥ 1.0	DOCX paragraph extraction in Stage 1 ingestion pipeline.
<code>pandas</code>	≥ 2.0	Loading and querying JSON/CSV mapping files for the semantic routing stage.
<code>numpy</code>	≥ 1.24	Cosine similarity computation between control embeddings and framework centroid vectors.

7.1 Setup Instructions

9. Clone the repository and navigate to the project root.
10. Create and activate a virtual environment: `python -m venv .venv && source .venv/bin/activate` (Linux/macOS) or `.venv\Scripts\activate` (Windows).
11. Install all dependencies: `pip install -r requirements.txt`
12. On first run, the sentence-transformer model will be downloaded automatically to the local Hugging Face cache (`~/.cache/huggingface/`). Subsequent runs are fully offline.
13. Launch the application: `streamlit run app.py`
14. The application will be available at `http://localhost:8501` by default.



Offline Deployment

For air-gapped or offline deployment, pre-download the transformer model to a local directory and set the `TRANSFORMERS_OFFLINE=1` and `HF_DATASETS_OFFLINE=1` environment variables. Point `MODEL_NAME` in `transform.py` to the local model path.

8. Known Limitations and Future Roadmap

8.1 Known Limitations

- **Memory pressure on large documents:** The embedding model is loaded into process memory once at startup. Very large documents submitted for validation may cause memory pressure on systems with limited RAM, as the full text is held in memory during encoding.

- **Complex DOCX formatting:** DOCX files containing complex layout elements — tables, nested content controls, non-standard styles, or heavily formatted paragraph runs — may produce incomplete or out-of-order text extraction through `python-docx`. Plain-prose DOCX files are handled reliably.
- **Static centroid vectors:** The mapping file centroid vectors are pre-computed and static. They do not learn from user submissions, feedback, or evolving framework language. Drift in compliance terminology over time will reduce routing accuracy until centroids are manually regenerated.
- **Single-control processing:** The current pipeline processes one control statement per validation run. Batch processing of multiple controls (e.g., from a full control catalog) requires multiple sequential runs through the UI.
- **English-only:** All normalization logic, keyword matching, obligation language detection, and mapping file content are English-language only. Non-English control statements will produce unreliable or failing results.

8.2 Future Roadmap

Feature	Description
Incremental centroid updates	Add a feedback loop allowing validated controls to incrementally update mapping file centroid vectors, improving routing accuracy over time without full recomputation.
Multi-control batch processing	Support CSV upload of multiple control statements, producing a batch results table with per-control scores, grades, and rationale — suitable for full control catalog assessment.
Confidence interval display	Surface the cosine similarity score alongside the framework assignment and display a confidence range alongside the compliance score to communicate evaluation certainty to the user.
Plug-in criterion architecture	Refactor the evaluation engine to support framework-specific criterion plug-ins, allowing each framework to define its own evaluation checklist without modifying core evaluation logic.

Feature	Description
Audit trail export	Generate a signed, timestamped audit record of each validation run — including all intermediate pipeline outputs — for use in formal compliance evidence packages.
Multi-language support	Extend sanitization, normalization, and keyword detection to support at minimum French and Spanish control statement language, aligned with international GRC deployment contexts.

📘 Contributing

Contributions to any roadmap item are welcome. Please refer to `README.md` for branching conventions, test requirements, and pull request guidelines. All new pipeline stages or circuit breaker conditions must include unit tests covering both the pass and halt paths before merge.